数学与 CS 问答笔记

answer-note

0101 评审的三条结论，各自在真实项目里的对应写法：映射型 AF 的工业写法、RI 与维护者成对出现、防御性拷贝的三种合法路线（声明别名 / 出向拷贝 / 入向所有权）。

November 05, 2026

Contents

1. 0101 评审对照：真实项目怎么处理 AF / RI / Safety（Real-world Cases）｜问题 0101 5
   - 1.1. 费曼直觉 5
   - 1.2. 硬核机制 5
     - 1.2.1. AF 的形状跟着抽象值走 5
     - 1.2.2. RI 与维护者成对出现 5
     - 1.2.3. Safety 的三条合法路线 6
   - 1.3. 落到项目 6
   - 1.4. 陷阱与易错 6
   - 1.5. 关键点 6
   - 1.6. 变式检验 6
   - 1.7. 同类归类 7
   - 1.8. Go 语言机制补充 7
   - 1.9. 本次操作与当前状态 7
2. 0101 第二轮评审：三处已改对，剩两处收尾（Round 2） 9
   - 2.1. 第二轮交卷与判定 9
   - 2.2. 三处改动对应的真实样本 9
   - 2.3. 剩下两处收尾 9
     - 2.3.1. AF 的竖线后为空 9
     - 2.3.2. RI 的维护者仍缺 10
   - 2.4. Safety 承诺要与实现对齐 10
   - 2.5. 题 6 的契约（放行） 10
   - 2.6. 本次操作与当前状态 10
3. 0101 知识卡片（草稿，待本人确认） 11
   - 3.1. 卡片 11
   - 3.2. 可迁移原则 11
   - 3.3. 反例 11
   - 3.4. 语言落点 11
   - 3.5. 6.031 对照 11
4. 0101 真实项目怎么用：四个判断点的工业样本（Real-world Usage） 12
   - 4.1. 规则 12
   - 4.2. 判断点一：未命中用什么表达 12
   - 4.3. 判断点二：Set 的返回值 12
   - 4.4. 判断点三：切片返回值的三条路线 12
   - 4.5. 判断点四：真实项目里的不变式注释 13
   - 4.6. 本次操作与当前状态 13
5. 题 5 升级：testing strategy（6.102 R02）｜问题 0101 14
   - 5.1. 为什么补这一题 14
   - 5.2. 6.102 R02 的原文要求 14
   - 5.3. Go 里的对应物 14
   - 5.4. 本项目要写的内容 14

5.5. 脚手架位置 15
5.6. 本次操作与当前状态 15
6. 维度与子域：从 6.102 R02 到 KV（Partitions and Subdomains） 16
6.1. 原文定义 16
6.2. 三个例子 16
6.2.1. abs：一维，两个子域 16
6.2.2. max：一个 partition，三个子域 16
6.2.3. KV：三个 partition 交叉 16
6.3. 真实项目怎么写 17
6.4. 回填到脚手架 17
6.5. 本次操作与当前状态 17
6.6. 第一处填写的评测（partition on key state） 17
6.6.1. operation 维度怎么切（IntSet 示例） 18
6.7. 本次操作与当前状态 18
7. 笛卡尔积与多分区：什么时候会缩小（Cartesian Product vs Multiple Partitions） . 19
7.1. 6.102 R02 的原文 19
7.1.1. 笛卡尔积的代价 19
7.1.2. 替代：多个独立 partition 19
7.1.3. 互斥组合会让笛卡尔积变小 19
7.2. 判定：直觉对的部分与要修的两点 19
7.3. 三种节省机制 19
7.4. 映射到 KV 20
7.5. 真实项目样本 20
7.6. 本次操作与当前状态 20
8. 第二处填写的评测（partition on operation） 21
8.1. Set 那一行为什么不成立 21
8.2. 真实项目怎么按操作组织 21
8.3. 还缺与格式 22
8.4. 本次操作与当前状态 22
9. 今日 token 用量异常诊断（2026-11-05） 23
9.1. 费曼直觉 23
9.2. 硬核机制 23
9.3. 陷阱与易错 24
9.4. 关键点 24
9.5. 变式检验 24
9.6. 同类归类 24
9.7. 本次操作与当前状态 24
10. 0101 第三轮评审：Set 分叉点改成提示，三块仍未落子（Round 3） 26
10.1. 交卷内容与判定 26
10.2. Set 的三个子域怎么写 27
10.3. 真实项目怎么表达「键是否已存在」与「值是否变了」 27
10.4. 还缺的三块 28

10.4.1. partition on history 怎么写 28
10.4.2. 7 条断言各自覆盖哪个子域 28
10.4.3. 未覆盖子域清单（阶段末判定） 28
10.5. 本次操作与当前状态 29
11. 用 Delve 调试 kv_test.go 里的测试（dlv test 实战） 30
11.1. 费曼直觉 30
11.2. 硬核机制 30
11.2.1. dlv test 到底做了什么 30
11.2.2. 命令速查 30
11.2.3. 实测：在失败断言行停下来 31
11.2.4. 条件断点 32
11.2.5. 非交互、headless 与 IDE 32
11.3. 陷阱与易错 33
11.4. 关键点 33
11.5. 变式检验 33
11.6. 同类归类 34
11.7. Go 语言机制补充 34
11.8. 本次操作与当前状态 35
Index of Listings 36

1. 0101 评审对照：真实项目怎么处理 AF / RI / Safety（Real-world Cases）｜问题 0101

1.1. 费曼直觉

课程给的是判据，工业代码给的是「判据落地后的形状」。三条结论各有一个可打开的真实样本：

- AF 写成映射还是集合，取决于抽象值的形状——`CharSet` 是集合，`Header` / `Values` 是映射。
- RI 与它的维护者必须成对出现——教材里那个 `add` 只会 `data.add(x)` 的 `IntSet` 就是反例。
- Safety 不是「一定要拷贝」，而是**三条合法路线选一条并写清楚**：声明别名、出向拷贝、入向所有权。

> 📌 **核心要点** 读真实代码的价值不在抄写法，而在看清「同一个原则有几种可接受的落地方式」。评审时要说的是「你选了哪条路线、有没有写清楚」，而不是「你必须拷贝」。

1.2. 硬核机制

1.2.1. AF 的形状跟着抽象值走

ADT	抽象值	AF 写法
`CharSet`（6.031 R11）	字符集合	`AF(s) = {s[i] | 0 <= i < s.length()}`
`KV`（本项目）	键到值的映射	`AF(mem) = { k -> mem[k] | 条件 }`
`ByteView`（group-cache）	不可变字节视图	自然语言：包裹 `[]byte` 或 `string`，细节对调用者不可见

`ByteView` 的注释还写明 `b` 与 `s` 的二选一关系——那不是类型保证，而是真 RI，必须写。

1.2.2. RI 与维护者成对出现

```
1    // 反例：RI 写了，但没有维护者
2    // rep invariant: data has no duplicates
3    private List<Integer> data = new ArrayList<>();
4    public void add(Integer x) { data.add(x); }
5
6    // 修法：把维护者写进变值器
7    public void add(Integer x) {
8        if (!data.contains(x)) data.add(x);
9    }
```

Listing 1: UWCSE 331 的 IntSet：RI 与 add 里的检查必须成对

Go 侧对照：`net/http` 的 `Header` 是 `map[string][]string`，注释里从不写「key 不重复」——类型已经保证，写了是噪音。

1.2.3. Safety 的三条合法路线

路线	真实样本	代价
声明别名 + 生命周期	`bytes.Buffer.Bytes()`：切片在下次修改前有效，改动会影响后续读	零拷贝，但调用者必须读懂文档
出向拷贝	`groupcache.ByteView.ByteSlice()`：返回 clone	每次读都分配
入向所有权声明	`groupcache.Sink.SetBytes`：the caller retains ownership of v	实现自己拷贝，调用者自由复用

⚠ **陷阱预警** `net/textproto` 的 `MIMEHeader.Set` 直接 `h[key] = []string{value}` ——因为 `string` 不可变，连拷贝都不需要。换成 `[]byte` 就必须显式处理，这正是 `KV.Set` 的处境。

1.3. 落到项目

评审结论	真实项目怎么处理	对本项目的直接含义
AF 抽象值形状不对	6.031 CharSet 用集合因为抽象值是集合；groupcache 用自然语言写表示与二选一	KV 抽象值是映射，必须写成 `{ k -> mem[k] | ... }`，并覆盖空 map
RI 缺维护者	331 IntSet：RI 与 `add` 里的 `contains` 成对	「key 非空」要么在 `Set` 检查，要么降级为前置条件
Safety 只写一半	Buffer 声明别名；ByteView 拷贝；Sink 声明所有权	写出向 + 入向两条，或明确选择文档声明路线

1.4. 陷阱与易错

- 把集合型 ADT 的 AF 模板套到映射型 ADT 上，丢掉「键对应值」。
- 把语言/类型保证（map 键唯一、切片长度可查）抄进 RI，造成「写了但没人维护」的假象。
- 只防出向别名，忘了 `Set` 存的是调用者的切片。
- 以为「标准库都拷贝」—— `bytes.Buffer.Bytes()` 恰恰不拷贝，靠文档约束。

1.5. 关键点

1. AF 的形状由抽象值决定：集合型用集合，映射型用映射。
2. RI 的判据是「能否写成检查函数」+「谁维持」。
3. Safety 有三条合法路线，选一条并写清楚，比盲目拷贝更专业。
4. 类型保证不该进 RI；语言特性（string 不可变）会改变安全写法。

1.6. 变式检验

把三条结论套到 `IntSet`（`elems []int`，升序无重复）：

- AF：`AF(elems) = { x | x 出现在 elems 中 }`，`[]` 代表空集；
- RI：升序且相邻不等（真 RI，因为切片允许乱序重复）；「元素总数不超容量」若从不检查 → 前置条件；
- Safety：`Elems()` 返回拷贝（出向）；`Add(xs ...int)` 若直接 `append`，调用者之后改 `xs` 会穿透（入向）。

1.7. 同类归类

概念	6.102	6.031
C09 AF/RI	R07 Abstraction Functions and Rep Invariants	R11 Invariants
C07 别名	R05 散见	R08 Mutability and Immutability
C05 规格	R04 Specifications	R06 Specifications

1.8. Go 语言机制补充

```
1    go doc -src bytes.Buffer.Bytes      # 出向：声明别名，不拷贝
2    go doc -src net/textproto.MIMEHeader.Set   # 入向：新建 []string，不借用调用
者切片
3    go doc -src net/url.Values.Get      # map 型 ADT 的观察器
4    go doc reflect.SliceHeader          # 切片头三字段 Data / Len / Cap
```

Listing 2: 本机可直接验证的四个出处

标准库对 `string` 与 `[]byte` 的区别对待，根因是 `string` 不可变：不可变的可以共享，可变的一旦共享就有别名风险。

1.9. 本次操作与当前状态

本次做了什么

- 读 `trialofcode.org/database/0101_kv_interface/`（作者讲义）：确认本步任务为 `Get` / `Set` / `Del`，`Open` / `Close` 是给定框架。
- 用 `go doc -src` 取标准库样本：`bytes.Buffer.Bytes`、`net/textproto.MIMEHeader.Set`、`net/url.Values.Get`。
- 检索 GitHub：`golang/groupcache` 的 `ByteView` 与 `Sink.SetBytes`，作为出向拷贝与入向所有权声明的工业样本。
- 检索 MIT 6.031 R11 与 UWCSE 331 的 `CharSet` / `IntSet` 写法，作为 AF 形状与 RI 维护者的教材样本。

验证结果

- `go doc -src` 三条均在本机成功返回源码；`bytes.Buffer.Bytes` 的注释明确写出「aliases the buffer content」。
- 未改动 `internal/db`：`kv.go` 仍是 00:15 那一版规格，`Get` / `Set` / `Del` 仍为 `panic`。

需要本人动手

1. AF 改为映射写法并覆盖空 map；

2. RI 决定「key 非空」的维护者（`Set` 检查 / 前置条件 / 删除），删除「not repeat」；
3. Safety 补入向拷贝一句（或明确选「声明别名」路线并写清生命周期）；
4. 补题 5 分区表，然后做题 6 实现。

2. 0101 第二轮评审：三处已改对，剩两处收尾（Round 2）

2.1. 第二轮交卷与判定

第二版与第一版的差异：

项	第一版	第二版	判定
AF	the set of keys that is unique each other and values paires	`{ k -> mem[k] | }`	🟩 改成映射写法
RI	三条，含 `not repeat`	两条，`not repeat` 已删	🟩 删对了
Safety	只写 `Get` 做防御性拷贝	`Get()` 与 `Set()` 都做	🟩 双向都写了

> 📌 **核心要点** 三处都改在了点上：AF 换成映射、RI 去掉类型保证、Safety 补上入向。剩下的两处不属于「错」，属于「没写完」——AF 的条件为空、RI 没写维护者。

2.2. 三处改动对应的真实样本

本次改动	真实项目样本	要点
AF 改成 `k -> mem[k]`	`net/url.Values` 是 `map[string][]string`，`Get` 从映射取值	抽象值形状必须是映射
删掉 `not repeat`	`net/http.Header` 同为映射，注释从不写「key 不重复」	类型保证不进 RI
`Set` 也加拷贝	`groupcache.Sink.SetBytes` 的 caller retains ownership of v	入向所有权声明

2.3. 剩下两处收尾

2.3.1. AF 的竖线后为空

```
AF(mem) = { k -> mem[k] | }        // 条件缺失，读起来像写了一半
```

Listing 3: 第二版 AF 的现状

两种补法：

```
// 选择 A：写全条件，与 RI 保持一致
AF(mem) = { k -> mem[k] | k ∈ keys(mem) 且 len(k) > 0 }

// 选择 B：用文字描述，同时覆盖空 map
AF(mem) = 把 mem 看作「非空字节串键 → 字节串值」的映射；为空时抽象值为空映射
```

Listing 4: 补全 AF 的两种写法

2.3.2. RI 的维护者仍缺

> ⚠️ **陷阱预警** `len(key) >= 1` 与 `len(value) >= 1` 写进了 RI，但 `Set` 里没有任何检查。按 6.102 R07 的判据，RI 必须由实现维持；不维持就必须降级为前置条件。**既不实现也不标注**，是最糟的第三种状态。

三个合规方案：

方案	写在哪	代价
A	`Set` 里检查，违反返回 `error`	与当前测试不冲突；RI 真正成立
B	注释写 `requires: len(key) > 0 && len(val) > 0`	义务交给调用者，RI 里不再保留这两条
C	标 `TODO(阶段末)`，本阶段不处理	符合「强度跟随测试」，但要承认这两条此刻是空话

2.4. Safety 承诺要与实现对齐

```
// 你写的 Safety: Get() 与 Set() 都做防御性拷贝

// 若实现成下面这样，注释与代码就矛盾了
func (kv *KV) Set(key, val []byte) (bool, error) {
  kv.mem[string(key)] = val   // 没有拷贝
  return true, nil
}

// 两条合规路线之一：
//   路线 1: 真的拷贝（bytes.Clone）
//   路线 2: 注释改成「声明别名 + 生命周期」（参照 bytes.Buffer.Bytes）
```

Listing 5: Safety 的承诺必须与实现一致

2.5. 题 6 的契约（放行）

```
Get: 命中 → (val, true, nil)；未命中 → (nil, false, nil)
Set: 新键 → updated = true；同键同值 → updated = false（讲义要求，测试不验）
Del: 键存在 → (true, nil)；键不存在 → (false, nil)
```

Listing 6: 三个方法的最小契约

2.6. 本次操作与当前状态

- 读取 `internal/db/kv.go` 第二版（823 字节，2026-11-05 16:46 修改）：AF / RI / Safety 三处已按第一轮评审修改。
- 未改动 `internal/db` 任何文件。
- `Get` / `Set` / `Del` 仍为 `panic("TODO: 0101")`；题 5 分区表尚未补。
- 下一步：补 AF 条件与空 map 一句、给 RI 选维护者方案、按契约实现三个方法，然后跑 `go test -count=1 ./internal/db/` 。

3. 0101 知识卡片（草稿，待本人确认）

3.1. 卡片

面	内容	依据
正面	`Get` 未命中时返回什么？为什么不是 `error`？	`kv_test.go:22`
背面	`(nil, false, nil)`：查不到是正常语义；`error` 只表达真正的失败	6.102 R04 / 6.031 R06
正面	`Set` 的 `updated` 表示什么？	作者讲义 0101
背面	状态是否真的变了：新键 `true`、同键同值 `false`；不是「成功了吗」	讲义「三个接口设计上的取舍」
正面	防御性拷贝要防几个方向？	C07
背面	两个：出向（`Get` 返回值）与入向（`Set` 收到的切片）	`groupcache.Sink.SetBytes`

3.2. 可迁移原则

1. 未命中是语义结果，不是失败；返回值表达语义，`error` 表达失败。
2. `AF` 的抽象值形状决定写法：映射型 ADT 用 `{ k -> v | ... }`，集合型才用集合。
3. `RI` 必须由实现维持；类型保证（map 键唯一）不进 `RI`。
4. 防御性拷贝分入向与出向两条路径。

3.3. 反例

- 用 `errors.New` 表达未命中：`kv_test.go:22`、`kv_test.go:31` 会红。
- `Set` 总返回 `true`：讲义要求的「同键同值 → `false`」用例会红——但当前官方测试不覆盖这条，所以这个 bug 现在看不见。
- `Del` 忘了先查存在性：返回值恒为 `true`，`kv_test.go:25` 会红。

3.4. 语言落点

```
1    val, ok := kv.mem[string(key)]   // comma-ok：未命中 ok=false，val=nil
2    delete(kv.mem, string(key))       // 无返回值，删没删到不告诉你
3    bytes.Equal(old, val)             // 切片比较；== 对切片不合法
```

Listing 7: 三个本步会用到的 Go 机制

3.5. 6.031 对照

同一步在 6.031 对应 R06（规格）、R11（AF/RI）、R08（可变性）；6.102 对应 R04、R07、R05。差异：6.031 用 Java 的 `@throws` 表达失败，Go 用 `(T, error)` 的返回值，未命中则走 `ok bool` 而不是异常。

4. 0101 真实项目怎么用：四个判断点的工业样本（Real-world Usage）

4.1. 规则

本人 2026-11-05 要求：讲解概念与给出评测意见时，必须附真实项目的用法样本（片段 + 出处），不需要全量代码，能看懂真实使用情况即可。本规则已写入 `AGENTS.md`。

4.2. 判断点一：未命中用什么表达

标准库 `sync.Map`（本机 `go doc -src sync.Map.Load`）：

```
1    // Load returns the value stored in the map for a key, or nil if no
2    // value is present. The ok result indicates whether value was found in
the map.
3    func (m *Map) Load(key any) (value any, ok bool)
```

Listing 8: 标准库选择 ok bool：未命中不是错误

bbolt 的 `Bucket.Get` 文档写「Returns a nil value if the key does not exist」——用 nil 表达不存在，连 `ok` 都不要。

项目	未命中的表达	对 error 的态度
`sync.Map`	`ok bool`	`error` 不参与
bbolt `Bucket.Get`	返回 `nil`	不区分「不存在」与「空值」
本项目 `KV.Get`	`ok bool` + `val == nil`	测试要求 `err == nil`

4.3. 判断点二：Set 的返回值

项目	签名	表达什么
标准库 map	赋值语句，无返回值	什么都不表达
bbolt `Bucket.Put`	`Put(key, value []byte) error`	只报失败
本项目 `KV.Set`	`Set(key, val []byte) (updated bool, err error)`	状态是否真的变了

> 📌 **核心要点** `updated bool` 不是业界标配：bbolt 与标准库都不这么写。它的理由在后续步骤——0103 的日志只有在状态真变时才写，0305 的 SQL 影响行数靠它计数。作者讲义把这条动机写得很清楚，属于**为后续步骤预留的接口**。

4.4. 判断点三：切片返回值的三条路线

项目	做法	约束方式
bbolt	不拷贝，返回库内内存	文档声明：仅在事务生命周期内有效
`bytes.Buffer.Bytes`	不拷贝，返回 `b.buf[b.off:]`	文档声明：下次修改前有效
`groupcache.ByteView.ByteSlice`	拷贝	调用者随便改

本项目的 Safety 写的是「`Get()` 与 `Set()` 都做防御性拷贝」，属于第三条路线；实现必须与之一致。

4.5. 判断点四：真实项目里的不变式注释

真实 Go 项目几乎不写 `AF` / `RI` 术语，但 `Invariant:` 注释很常见（本机 GOROOT 可搜到 20 余处）：

```
// compress/flate/dict_decoder.go
// Invariant: 0 <= rdPos <= wrPos <= len(hist)

// runtime/mgcwork.go
// Invariant: Both wbuf1 and wbuf2 are nil or neither are.

// internal/poll/splice_linux.go
// Invariant: when entering spliceDrain, the pipe is empty.
```

Listing 9: GOROOT 里的三处 Invariant 注释

> 📌 **核心要点** 这三条都是**能被检查的表示关系**：区间大小、两字段同时为 nil、进入函数时的状态。没有一条属于「map 的键不重复」这种类型保证——这正是本项目 RI 删掉 `not repeat` 的依据。

4.6. 本次操作与当前状态

- 在 `AGENTS.md` 新增「讲解与评测必须附真实项目样本」一节（2026-11-05 用户要求）。
- 用 `go doc -src sync.Map.Load` 取标准库样本；用 `rg` 在 GOROOT 搜到 20 余处 `Invariant:` 注释；bbolt 的文档原文取自 pkg.go.dev。
- 未改动 `internal/db`：`kv.go` 的 `Get` / `Set` / `Del` 仍为 `panic("TODO: 0101")`。
- 下一步仍是题 6 实现，实现后跑 `go test -count=1 ./internal/db/` 验收。

5. 题 5 升级：testing strategy（6.102 R02）｜问题 0101

5.1. 为什么补这一题

6.102 R02 的 test-first 包含三件事：写 testing strategy、写测试用例、然后实现。原题单只给了「补两个子域」，把写完整 strategy 这一半压掉了——这一节把它补回来。

5.2. 6.102 R02 的原文要求

原文（`6.102 classes/02-testing` 的 **Documenting your testing strategy** 小节）：

> write down the testing strategy you used to create a test suite: the partitions, their subdomains, and which subdomains each test case was chosen to cover.

它给的最小格式：

```
/*
 * Testing strategy
 *
 * partition:
 *   a < b
 *   a > b
 *   a = b
 */
```

Listing 10: 6.102 原文里 max 函数的 strategy 写法

多分区时原文明确区分两种写法：

- `cover the cartesian product of these partitions`：多个分区自由组合；
- `cover the subdomains of these partitions`：只覆盖有意义的组合。

用例要标注它覆盖的子域，原文示例是 `it("covers a < b", ...)`。

5.3. Go 里的对应物

6.102 / TypeScript	Go	出处
`describe(...)` 顶部的块注释	`TestXxx` 上方块注释	社区惯例，Go 无强制格式
it 子测试命名	t.Run(name, fn) 子测试	go doc testing.T.Run
表驱动用例	`[]struct{ name string; ... }` + `t.Run`	GOROOT `src/strconv/strconv_test.go` 的 `mallocTest`

标准库 `strconv/strconv_test.go` 里每个表项都带一个 `desc` 字段说明覆盖的场景，这就是「用例标注子域」在 Go 里的落地方式。

5.4. 本项目要写的内容

项	要求	判据

partitions	至少三个维度：操作 / 键状态 / 调用历史	维度之间互不重叠
子域	每个分区拆到互斥子域，含边界	空键、同值、异值这些边界要出现
用例映射	7 条断言各自覆盖哪个子域	`kv_test.go:12/16/19/22/25/28/31` 每条都能指到
缺口	当前未覆盖的子域	同键同值、同键异值、空键、别名至少列出

5.5. 脚手架位置

- `internal/db/kv_test.go` 顶部：`Testing strategy` 块注释，全部为 `TODO` 占位；
- `internal/db/kv.go`：三个方法体各留分步 `TODO` 注释，末尾仍是 `panic("TODO: 0101")`。

5.6. 本次操作与当前状态

- 给 `kv_test.go` 与 `kv.go` 加了脚手架（待填），`go vet ./internal/db/` 通过。
- `go test -count=1 ./internal/db/` 的红基线：`panic: TODO: 0101`，栈顶 `kv.go:38`（`Set`），由 `kv_test.go:43` 调用——脚手架没有改变红基线位置。
- 待本人填写：strategy 全文 + `Get` / `Set` / `Del` 三个方法体。

6. 维度与子域：从 6.102 R02 到 KV（Partitions and Subdomains）

6.1. 原文定义

6.102 R02 的原文：

> we divide the program's input space into subdomains, each consisting of a set of inputs. Taken together, the subdomains form a partition. A partition is a collection of non-empty disjoint sets that completely covers the input space, so that every input lies in exactly one subdomain.

三个硬条件：**非空**、**互不重叠**、**合起来覆盖全部合法输入**。满足这三条的切法才叫 partition。

> 📌 **核心要点** 口语里的「维度」= 一个 partition（一种切法）；「子域」= 这种切法下的一个互斥取值类别。所以「写子域」不是写函数名，而是写这个函数的行为被分成几类。

6.2. 三个例子

6.2.1. abs：一维，两个子域

```
1    // partition: a >= 0; a < 0
```

Listing 11: 6.102 原文的 abs 例子

子域 `a >= 0` 的行为是原样返回，`a < 0` 是取反——行为不同的输入必须切开。

6.2.2. max：一个 partition，三个子域

```
1    // partition: a < b; a > b; a = b
```

Listing 12: 6.102 原文的 max 例子

只写 `a < b; a > b` 是不够的：`a = b` 的输入没被覆盖，那就不叫 partition。**覆盖完整**是硬要求。

6.2.3. KV：三个 partition 交叉

PARTITION（维度）	子域
operation	Get / Set / Del
key state	存在 / 不存在
history	初始 / 删除后

组合起来：`kv_test.go:28` 的「删存在的键」落在 `(Del, 存在, 初始)`；`kv_test.go:31` 的「删除后再读」落在 `(Get, 不存在, 删除后)`——后者只有引入 history 维度才描述得出来。

6.3. 真实项目怎么写

标准库 `/usr/lib/go/src/strconv/number_test.go` 的 `TestFormatComplex`：

```
    tests := []struct {
      c       complex128
      fmt     byte
      prec    int
      bitSize int
      out     string
    }{
      // a variety of signs
      {1 + 2i, 'g', -1, 128, "(1+2i)"},
      {-7 - 8i, 'g', -1, 128, "(-7-8i)"},

      // test that fmt and prec are working
      {3.14159 + 0.00123i, 'e', 3, 128, "(3.142e+00+1.230e-03i)"},
    }
```

Listing 13: 标准库表驱动：注释标出维度，每个 case 是一个子域的代表

每个注释块标出一个维度（符号、精度），每个 case 是该维度下一个子域的代表——这是「choose one test case from each subdomain」在 Go 里的落地形态。

6.4. 回填到脚手架

```
     *   partition on operation:
     *     Get: 命中 / 未命中
     *     Set: 新键 / 同键同值
     *     Del: 存在 / 不存在
```

Listing 14: 示意形态：冒号前是维度，冒号后是子域

6.5. 本次操作与当前状态

- 本节为本人提问「维度的子域是什么意思」的回答，未改动 `internal/db`。
- `kv_test.go` 与 `kv.go` 仍是脚手架状态（`TODO` 占位 16 个 / 13 个），待本人填写分区与子域。

6.6. 第一处填写的评测（partition on key state）

本人已填：

```
     *   partition on key state: TODO
    *     key in mem and key not in mem
```

Listing 15: 当前内容（原文照录，含格式问题）

判定：内容正确——`key in mem` / `key not in mem` 互斥且覆盖全部键状态，是一个合格的 partition。

格式两处待修：

问题	现状	应改为
`TODO` 未删	`partition on key state: TODO`	`partition on key state:`
第 12 行顶格	`*  key in mem...`（`*` 在第 1 列）	`  *  key in mem...`（前导留一个空格）

6.102 R02 原文的紧凑写法是行内分号（`// partition: a < b; a > b; a = b`），也可以保留分行。

还缺的四块：`partition on operation` 的三个子域、`partition on history`、7 条断言的映射、未覆盖子域。

6.6.1. operation 维度怎么切（IntSet 示例）

场景：`IntSet` 存当天出现过的用户 ID，不重复、支持快速判断在不在。三个操作各自按「行为分叉点」切子域：

```
    *   partition on operation:
    *     Contains: element present; element absent
    *     Add:      new element; duplicate element
    *     Remove:   element present; element absent
```

Listing 16: 每个操作只按自己的行为分叉点切

映射回 KV：`Get` 关心「键在不在」，`Set` 关心「新键还是覆盖」，`Del` 关心「删到没删到」。三行形式照抄即可，内容自己判断。

> 📌 **核心要点** 切子域的判据只有一条：**这个操作的行为会不会因为输入落在不同类别而不同**。行为相同的输入归到一个子域，行为不同的必须切开。

6.7. 本次操作与当前状态

- 读取 `kv_test.go`（1497 字节，2026-11-05 17:32:14）：本人已填 `partition on key state` 一处，`TODO` 从 16 降到 15。
- 未改动 `internal/db` 任何文件。
- 待本人继续填 operation / history 与 7 条断言映射，填完后整篇评测。

7. 笛卡尔积与多分区：什么时候会缩小（Cartesian Product vs Multiple Partitions）

7.1. 6.102 R02 的原文

7.1.1. 笛卡尔积的代价

> forming a single partition of the input space from the Cartesian product of the behavior on each parameter leads to a combinatorial explosion in the size of the resulting test suite. We saw this already with BigInt multiplication, in which the Cartesian product partition already had 6 × 6 = 36 subdomains, requiring 36 test cases to cover. For a function with n parameters, the Cartesian-product approach produces a test suite of size exponential in n.

7.1.2. 替代：多个独立 partition

> Now we have three partitions, with 6, 6, and 5 subdomains each, but we don't need the Cartesian product of 6 × 6 × 5 test cases to cover them. A test suite with 6 carefully-chosen test cases can cover the subdomains of all three partitions.

7.1.3. 互斥组合会让笛卡尔积变小

> But even in those cases, the Cartesian product may be smaller than we expect. When subdomains from different partitions turn out to be mutually exclusive, the Cartesian product won't include a subdomain for that particular combination of subdomains.

7.2. 判定：直觉对的部分与要修的两点

说法	判定	依据
cartesian product 就是各 partition 子域的全组合	对	原文 6 × 6 = 36
实际很少真的全组合	对	6 个用例覆盖 6+6+5
子域互斥会让笛卡尔积变小	对	原文最后一段
「规格会缩小很多」	需修	缩小的是用例集，规格不变
「互斥或者相同」	需修	「相同」属行为等价，不是组合不存在

7.3. 三种节省机制

机制	6.102 的例子	效果
多个独立 partition 替代一个笛卡尔积	6+6+5 子域 → 6 个用例	指数 → 线性
剔除互斥组合	`a=0` 与 `b` 的符号类无法同时成立	笛卡尔积变小
一个用例覆盖多个子域	一个 `(a,b)` 同时覆盖 a、b 的子域	用例数减少

> 📌 **核心要点** 同一个 partition 内的子域天然互斥（这是 partition 的定义）；**跨** partition 的子域才可能互斥。行为等价（不同组合走同一条代码路径）属于第三种机制，不是互斥。

7.4. 映射到 KV

```
partition on operation: Get / Set / Del        (3)
partition on key state: in mem / not in mem   (2)
partition on history:  initial / after delete  (2)
=> 全组合 3 x 2 x 2 = 12
```

Listing 17: 三个 partition 的笛卡尔积是 12 组，7 条断言只覆盖其中一部分

`Get × key not in mem` 在「初始」与「删除后」两种历史下行为相同（都 `!ok`），属于行为等价，一个用例即可代表。

7.5. 真实项目样本

标准库 `strconv/number_test.go` 的 `TestFormatComplex`（本机可看）：整个文件 26 个测试函数，这一个函数只用了约 10 个 case，按「符号 / 精度 / bitSize」几类挑代表，而不是把各维度的所有组合铺开。

7.6. 本次操作与当前状态

- 本节为本人提问的回答（cartesian product 是否为子域全组合、互斥或相同会不会缩小规格），未改动 `internal/db`。
- `kv_test.go` 仍为 1497 字节、`TODO` 15 个（只填了 `partition on key state`）。

8. 第二处填写的评测（partition on operation）

本人第二次提交（1542 字节，2026-11-05 19:28:54）：

```
    *   partition on operation:
    *     Get: use key state
    *     Set: val in mem[k] and val not in mem[k]
    *     Del: use key state
```

Listing 18: partition on operation 的当前内容

行	内容	判定	说明
Get	use key state	🟩	行为确由键在不在决定；建议展开成「key in mem / key not in mem」
Set	val in mem[k] and val not in mem[k]	🟥	有实质错误，见下
Del	use key state	🟩	同 Get，可展开

8.1. Set 那一行为什么不成立

① **`val` 不是 `mem` 的键**：`mem` 的类型是 `map[string][]byte`，键是 `string(key)`，值是 `[]byte`。所以「`val` in `mem[k]`」这个说法不成立。

② **Set 的分叉点是两层**：

分叉点	子域	UPDATED
key 是否已存在	新键	true
已存在时值是否变化	同键同值	false（作者讲义明确要求）
已存在时值是否变化	同键异值	true

> 📌 **核心要点** 决定 `updated` 的是「**键**是否存在」与「**值**是否变了」，不是「值在不在」。作者讲义原文：`Set` 的 `updated` 表示「状态是否真的变了」——新键 `true`，同键同值 `false`。

8.2. 真实项目怎么按操作组织

标准库 `/usr/lib/go/src/sync/map_test.go` 没有用 `t.Run`，而是**按操作与场景拆成不同测试函数**：

```
func TestMapMatchesRWMutex(t *testing.T)         // 读写一致性
func TestMapMatchesDeepCopy(t *testing.T)        // 值拷贝
func TestMapRangeNestedCall(t *testing.T)        // 遍历中回调再遍历
func TestMapRangeNoAllocations(t *testing.T)     // 遍历零分配
func TestMapClearOneAllocation(t *testing.T)     // Clear 零分配
```

Listing 19: 标准库用函数名当子域标签

这就是 `partition on operation` 的另一种落地：每个操作或性质一个函数，函数名即子域标签。本项目只有一个 `TestKVBasic`，只能用注释标注，两者等价。

8.3. 还缺与格式

- 还缺三块：`partition on history`、7 条断言映射、未覆盖子域；
- 第 12 行 `partition on key state: TODO` 的 `TODO` 未删；
- 第 13 行 `*     key in mem...` 顶格，缺一个前导空格。

8.4. 本次操作与当前状态

- 读取 `kv_test.go`（1542 字节，19:28:54）：本人已填 `partition on key state` 与 `partition on operation`，`TODO` 从 15 降到 11。
- 未改动 `internal/db` 任何文件。
- 待本人修正 `Set` 子域、补 history / 映射 / 未覆盖三块。

9. 今日 token 用量异常诊断（2026-11-05）

9.1. 费曼直觉

把每次模型调用想成「把整本对话笔记复印一份，再问一句新问题」。今天真正的新问题只有几十句，但复印件从 42 页涨到 786 页，而且复印了 1176 次——用量自然爆表。

所以是**两个乘数同时到顶**：复印多厚（上下文长度）× 复印多少次（请求次数）。只修一个不够。

9.2. 硬核机制

数据来源（本机可复现）：`~/.reasonix/stats/<日期>.jsonl` 每次模型调用一行，带 `prompt` / `completion` / `cache_hit` / `cache_miss`；会话侧 `~/.reasonix/projects/<项目>/sessions/<会话>.wire.jsonl` 的 `usage` 与 `tool_result` 事件可按顺序归因「这次请求是被哪个工具触发的」。

第一张表：今日与其他日子的量级对比。

指标	11-04（昨日）	11-05（今日）	倍数
请求次数	176	1176	6.7×
输入 token	6,195,689	561,199,015	90×
输出 token	70,310	572,373	8×
平均每次输入	35,203	477,210	13.6×

第二个乘数来自一处会话：`20261102-144310` 这个会话从 11-02 14:43 开到现在没关，累计 1228 次请求、572.9M 输入 token。它的上下文从 42,741 token 单调涨到 786,080 token，`carriedMessages` 从 0 涨到 2484——也就是**每一次请求都把全部历史重新发一遍，中途没有压缩**。

第二张表：按「触发这次请求的上一个工具」归因整个会话的 572.9M token。

上一个工具	请求数	输入 token	占比
`bash` 轮询等文件变化	665	366,666,913	64.0%
`await_user` 交还控制	355	152,451,896	26.6%
`bash` 其它（测试/构建/查看）	110	27,528,028	4.8%
`edit_file`	36	8,904,096	1.6%
`read_file`	24	6,493,477	1.1%
`use_capability`	12	6,001,215	1.0%
其余（todo / search / ask / write）	24	4,808,336	0.8%

轮询命令就是同一条反复跑：

```
    ls -l --time-style=+%H:%M:%S internal/db/kv.go internal/db/kv_test.go;
date +%H:%M:%S; grep -c 'TODO' internal/db/kv_test.go
```

Listing 20: 等待本人改文件时反复执行的轮询命令

按轮拆开：turn11 跑 220 次、327 次请求 / 160.1M；turn12 跑 312 次、432 次请求 / 294.5M。两轮合计 454.7M，占今天的 81%。turn12 里还有一次 1,367,732 token 的单请求（`sa-2-1017`），是上下文叠加后的峰值。

9.3. 陷阱与易错

⚠️ **陷阱预警 把「token 多」误判成「回答太长」**。今天输出总共只有 57 万 token，占全部用量的 0.1%；99.9% 是输入重发。删减回答篇幅对这个问题几乎无效，真正要动的是**上下文长度**和**请求次数**。

把高 cache 命中当成免费。今天 cache 命中率 99.7%，计费打折，但 token 计数照算全额——你看到的使用量统计不会因为 cache 而变小。

⚠️ **陷阱预警「一步一小讲」不是元凶**。规则要求的长回答只贡献了 0.1% 的输出；把它当原因去改，会把课程规范改坏，而用量纹丝不动。

9.4. 关键点

📌 **核心要点** 今天的 561M 输入 token 里，约 90.6% 花在「等本人动手」上：64.0% 是一条轮询命令空转，26.6% 是 `await_user` 交还控制后再被触发。真正干活的工具（测试、构建、编辑、读文件、检索）合计不到 10%。这不是「做了什么多」，而是「等的时候还在烧」。

- 会话跨度 4 天、上下文 786K，是第一个乘数；
- 回合内轮询等文件、每个回合几百次请求，是第二个乘数；
- 两个乘数相乘，才有了 90 倍于昨日的用量。

9.5. 变式检验

- **只修上下文**：同样 1176 次请求，若上下文一直是开局那 42K，约 $1176 \times 42\text{K} \approx 49\text{M}$，可省约 91%。
- **只修轮询**：去掉 665 次轮询与 355 次 `await_user`，剩约 208 次请求，按平均 700K 算约 146M，仍比昨天高 23 倍——所以两件事都要做。
- **更好的解法**：一步一回合、一讲一会话。每步结束就把结论落盘进本笔记，下个会话只读笔记，不背全部历史。

9.6. 同类归类

同一形态的风险点在别的会话也能看到：家目录那批会话里 `20261031-140440` 累计 26M、`20261031-082812` 累计 11.3M，同样是「长会话 + 反复试探」的形状，只是没到今天的量级。共性是：**会话不重启、回合不长刀直入**。

9.7. 本次操作与当前状态

- 只读诊断，未改动 `internal/db` 任何文件。

- 已按本人决定在 `AGENTS.md` 新增「会话成本纪律」一节：禁止轮询等待、一步一回合、上下文 200K 上限、成本自查方法。
- 统计口径：`stats/2026-11-05.jsonl` 的 1176 条记录；会话归因用 `wire.jsonl` 的 1228 条 `usage` 事件。
- 下一步待本人执行：新开会话继续 0101，旧会话（上下文 786K）不再沿用。

10. 0101 第三轮评审：Set 分叉点改成提示，三块仍未落子（Round 3）

10.1. 交卷内容与判定

本人第三次提交（`internal/db/kv_test.go`，`TODO` 3 个）：

```
    * cover the cartesian product of these partitions:
    *   partition on operation:
    *     Get: use key state
    *     Set: TODO（分叉点：键是否已存在 / 值是否变化）
    *     Del: use key state
    *   partition on key state:
    *     key in mem
    *     key not in mem
```

Listing 21: operation 与 key state 两块的现状

项	第二轮	第三轮	判定	依据
Set 行	`val in mem[k] and val not in mem[k]`	`TODO（分叉点：...）`	🟨 退回到显式未完成	错误陈述消失，但子域还没写
Get / Del	`use key state`	同左	🟩 合法	引用另一个 partition，6.102 允许
缩进	第 13 行顶格	已有一致前导空格	🟩 已修	源码第 9 行起对齐
`partition on history`	`TODO`	`TODO`	🟥 未动	序列维度没切
7 条断言映射	缺	缺	🟥 未动	哪条断言覆盖哪个子域没写
未覆盖子域	`TODO`	`TODO`	🟥 未动	阶段末要判定的清单为空

📌 **核心要点** 把错误表述改回 `TODO` **是进步**：注释从「说错话」变成「承认没写」，评审不会再把错误当成结论。但 `TODO` 不是交付物——第三步必须把它换成真正的子域。

10.2. Set 的三个子域怎么写

Set 的行为由**两个**分叉点共同决定，但两者不是独立相乘：

键的状态	值的变化	UPDATED	测试现状
不存在（新键）	无可比较的旧值	true	已覆盖（第 36 行）
已存在	旧值 ≠ 新值	true	未覆盖
已存在	旧值 = 新值	false	未覆盖

「新键 × 同值」这一格**不存在**：没有旧值就没有「变了没变」。这正是上一节说的互斥组合——2 × 2 的笛卡尔积被剔除一格，只剩 3 个子域。

建议写法（三行，和 Get / Del 的层次对齐）：

```
    *   partition on operation:
    *     Get: key in mem; key not in mem
    *     Set: key absent; key present and value changed; key present and
value unchanged
    *     Del: key in mem; key not in mem
```

Listing 22: Set 的 3 个子域，名字直接对应行为分叉

⚠️ **陷阱预警** 不要把 Set 的子域写成「值在不在 map 里」。`mem` 的类型是 `map[string][]byte`，键是 `string(key)`、值是字节切片：**值本来就不是键**，所以「val in mem[k]」这种语句在类型上不成立，读注释的人也无法据此写出用例。

10.3. 真实项目怎么表达「键是否已存在」与「值是否变了」

标准库 `sync.Map` 把这两个分叉**拆成两个返回值**，各自带注释（本机可跑 `go doc -src sync.Map.LoadOrStore`）：

```
    // LoadOrStore returns the existing value for the key if present.
    // Otherwise, it stores and returns the given value.
    // The loaded result is true if the value was loaded, false if stored.
    func (m *Map) LoadOrStore(key, value any) (actual any, loaded bool)

    // CompareAndSwap swaps the old and new values for key
    // if the value stored in the map is equal to old.
    func (m *Map) CompareAndSwap(key, old, new any) (swapped bool)
```

Listing 23: sync/map.go 的两个样本：键存在性一个 bool，值是否变了另一个 bool

对照本项目：

问题	SYNC.MAP	KV.SET

键是否已存在	`loaded bool`	自己先取 `old, ok`
值是否变了	`swapped bool`（另一次调用）	`bytes.Equal(old, val)`
对外返回值	两个独立操作各自一个 bool	合成一个 `updated`

差异要有意识：标准库用**两个操作**覆盖两件事，本项目用一个 `Set` 覆盖两件事，所以 `Set` 的注释必须说明 `updated` 同时承担了「新增」和「真的变了」两种含义——这正是讲义要求「同键同值返回 false」的来源。

10.4. 还缺的三块

10.4.1. partition on history 怎么写

历史维度不参与笛卡尔积里的「参数组合」，它描述的是**调用顺序**。两种合法写法：

```
// 写法 A：当成序列维度
 *   partition on history:
 *     initial; after Set; after Del

// 写法 B：不叫 partition，直接列出要覆盖的交互
 * covers interactions: Set→Get; Set→Set; Del→Del; Set→Del→Get
```

Listing 24: history 的两种落法

10.4.2. 7 条断言各自覆盖哪个子域

断言行	覆盖的子域	历史
32	`Open` 成功	initial
36	Set × 新键	initial
39	Get × key in mem	after Set
42	Get × key not in mem	initial
45	Del × key not in mem	initial
48	Del × key in mem	after Set
51	Get × key not in mem	after Del

这张表一填，就能看出两件事：Set 的 3 个子域只测了 1 个；Del 之后没有人再调 Del。

10.4.3. 未覆盖子域清单（阶段末判定）

未覆盖	属于哪条规格	建议
Set × 旧值 ≠ 新值 / 旧值 = 新值	`updated` 的分叉	现在补，1 行断言
Del 后再 Del	history	现在补，1 行断言
Get 返回的切片改掉后 `mem` 不变	Safety（出向拷贝）	`TODO(阶段末)`
Set 后改入参切片 `mem` 不变	Safety（入向拷贝）	`TODO(阶段末)`

空键 / 空值	RI 的 `len >= 1`	`TODO(阶段末)`

> 📌 **核心要点** 判据是「规格强度跟随测试」：`updated` 和 history 两行**现在就该补**，因为讲义已经把行为写死；Safety 与 RI 三行属于阶段末议题，测试没明确之前只标 `TODO(阶段末)`，不提前加严。

10.5. 本次操作与当前状态

- 读取 `internal/db/kv_test.go`：`TODO` 从 11 降到 3，`Set` 行已从错误陈述改成显式提示，缩进已对齐。
- 跑 `go test ./internal/db/ -run TestKVBasic -v`：**红**。失败在 `kv_test.go` 第 36、39、42、45、51 行；第 32 行通过。
- 红的原因是 `internal/db/kv.go` 的实现未完成（`Get` 未命中返回了 `ErrNotFound`，`Set` 里 `make` 没 `copy`），**不是**测试策略的问题；这部分属于 0101 的题 6。
- 未改动 `internal/db` 任何文件。
- 待本人：把 `Set` 的三个子域写进去、补 history、补断言映射与未覆盖清单，然后跑一次 `go test -count=1 ./internal/db/` 确认红点只来自实现。

11. 用 Delve 调试 kv_test.go 里的测试（dlv test 实战）

11.1. 费曼直觉

`go test` 的样子是「跑完给一张成绩单」：哪一行断言失败、期望 true 得到 false，然后没了。变量当时是什么值、`val` 到底长了什么样，全看不到。

Delve 做的事只有一件：**把测试编成一个普通程序，然后把它按在某一行动不了**。你什么时候让它继续，它才继续。于是在这一行上，你可以像读一张快照一样读所有变量。

所以调试的核心不是记命令，而是想清楚一件事：**我应该在源代码的哪一行把它按住？**对测试而言，答案几乎总是「失败的那条断言行」。

11.2. 硬核机制

11.2.1. dlv test 到底做了什么

`dlv help test` 的原话：

```
    Compiles a test binary with optimizations disabled and begins a new debug
session.
    The test command allows you to begin a new debug session in the context
of your
    unit tests. By default Delve will debug the tests in the current
directory.
    Alternatively you can specify a package name, and Delve will debug the
tests in
    that package instead. Double-dashes `--` can be used to pass arguments to
the test program:

    dlv test [package] -- -test.run TestSomething -test.v -other-argument
```

Listing 25: dlv help test 的输出（本机 dlv 1.27.2）

「optimizations disabled」= 自动加 `-gcflags=all=-N -l`：`-N` 关优化、`-l` 关内联（定义在 `/usr/lib/go/src/cmd/compile/internal/base/flag.go` 第 64 行与第 74 行）。为什么必须关，见本节后面的对照实验。

本项目最小的启动方式：

```
    cd /home/zz/project/db-go
    dlv test ./internal/db -- -test.run TestKVBasic -test.v
```

Listing 26: 进入交互式调试；– 之后是传给测试二进制的参数

`-test.run` 的值是正则（`go help testflag` 原文：Run only those tests, examples, and fuzz tests matching the regular expression），所以 `-test.run 'TestKV.*'` 可以一次选中一组。

11.2.2. 命令速查

命令	作用	本项目里的用法
`break` / `b`	在行、函数或文件:行上设断点	`break kv_test.go:39`

`continue` / `c`	跑到下一个断点	`continue`
`next` / `n`	单步，不进入被调函数	越过 `kv.Get(...)` 那一行
`step` / `s`	单步，进入被调函数	从断言行钻进 `Get` 内部
`stepout`	执行完当前函数并返回调用处	看完 `Set` 后回测试
`print` / `p`	求值一个 Go 表达式	`print string(val)`
`locals` / `args`	局部变量 / 函数参数	`args -v` 看 `key`、`val`
`display`	每次停下都自动打印某表达式	`display string(key)`
`bt`	调用栈	确认是谁调了 `Get`
`goroutines`	列出协程，可切换	测试用了 `t.Parallel` 时
`list`	显示当前行附近的源码	`list Del`
`exit` / `quit`	结束会话并杀掉被调试进程	脚本最后一行

11.2.3. 实测：在失败断言行停下来

在 `kv_test.go` 第 39 行（`assert.True(t, string(val) == "v1" && ok && err == nil)`）打断点，读它当场看到的四个值：

```
    break kv_test.go:39
    continue
    print string(val)
    print ok
    print err
    print kv.mem
    print string(kv.mem["k1"])
    exit
```

Listing 27: 把上面的命令存成 init 文件，用 –init 喂给 dlv 即可复现

本机实际输出（节选，去掉源码上下文行）：

```
    Breakpoint 1 set at 0x696510 for github.com/zz-cpp/db-go/internal/
db.TestKVBasic() ./internal/db/kv_test.go:39
    "\x00\x00"
    true
    error nil
    map[string][]uint8 [
      "k1": [0,0],
    ]
    "\x00\x00"
```

Listing 28: 第 39 行停下时的 val / ok / err / mem

这张快照一次说明三件事：

- `ok` 是 `true`、`err` 是 `nil`，所以断言里 `ok && err == nil` 这一段**成立**；
- `val` 是两个零字节，`string(val) == "v1"` 不成立——失败点被精确定位到值的内容；
- `kv.mem["k1"]` 本身也是 `[0,0]`，说明问题不在 `Get` 的拷贝，而在 `Set` 存进去的就是零字节（`make` 申请了长度却没 `copy`）。

`go test` 只能告诉你「第 39 行 should be true」；delve 把「为什么」也给了出来。

11.2.4. 条件断点

`Get` 在测试里被调了三次（`k1`、`xxx`、`xxx`），每次都停很烦。Delve 的语法是「位置 + `if` + Go 表达式」：

```
break db.(*KV).Get if string(key) == "xxx"
continue
print string(key)
print ok
```

Listing 29: 只在键为 xxx 时停下；实测命中的是第三次调用

两个细节：

- 包名可以只写 `db`，Delve 会把它补全成 `github.com/zz-cpp/db-go/internal/db.(*KV).Get`；
- 条件表达式在**每次到达断点时求值**，所以断点位置必须让表达式里的变量已经可见（在函数入口处 `key` 参数已可读）。

> ⚠️ **陷阱预警** 不要模仿 shell 习惯把条件塞进位置参数。下面这种写法本机实测报错 `location ... not found`——Delve 把整段当成了位置名：
>
> `break db.(*KV).Get "string(key) == \"xxx\""`
>
> 条件必须用 `if` 引出：`break db.(*KV).Get if string(key) == "xxx"`。

11.2.5. 非交互、headless 与 IDE

场景	命令	本机实测
脚本化复现	`dlv test ./internal/db --init 命令文件 -- -test.run TestKVBasic`	本笔记所有输出都由它产生
非终端环境	追加 `--allow-non-terminal-interactive=true`	不加会直接拒绝启动
IDE 接入（DAP）	`dlv dap --listen=127.0.0.1:0`	输出 `DAP server listening at: 127.0.0.1:38343`
远程 / 复用会话	`dlv test ./internal/db --headless --listen=127.0.0.1:40000`	输出 `API server listening at: 127.0.0.1:40709`

远程模式下，另一个终端用 `dlv connect 127.0.0.1:40000` 接入同一个会话，断点与变量与本地一致。

VS Code 的 Go 扩展就是 DAP 客户端：`launch.json` 里 `"mode": "test"` 等价于这里的 `dlv test`，`"program"` 指向包目录，`"args"` 写 `-test.run` 等参数（字段以 `github.com/golang/vscode-go` 的 debugging 文档为准）。断点仍然落在源码行上，和命令行完全一致。

11.3. 陷阱与易错

⚠️ **陷阱预警 在函数入口处想 print 局部变量，会失败。**在 `db.TestKVBasic` 入口停下时执行 `print kv`，本机报 `could not find symbol value for kv` —— `kv := KV{}` 这一行还没执行，变量尚未进入作用域。先 `next` 越过声明，再 print。

⚠️ **陷阱预警 `locals` 不含参数。**在 `(*KV).Del` 入口处 `locals` 显示 `(no locals)`，而 `print string(key)` 正常返回 `"xxx"`。参数要用 `args`（`args -v` 连类型一起给）。

⚠️ **陷阱预警 未初始化的接口值读不出来。** `Get` 入口处 `print err` 输出 `(unreadable could not resolve interface type)`。这是 Delve 对零值 `error` 的限制，不是程序 bug；用 `err == nil` 这类表达式判断即可。

⚠️ **陷阱预警 用 `dlv exec` 调试 `go test -c` 的默认产物，断点会漂。**本机实测：断点设在 `db.(*KV).Del`，实际停在 `TestKVBasic`，并打印 `Warning: debugging optimized function`；`locals` 为空、`key` 找不到（因为 `Del` 被内联了）。修法是自己加编译标志：

```
go test -c -gcflags="all=-N -l" -o /tmp/db-n.test ./internal/db
dlv exec /tmp/db-n.test -- -test.run TestKVBasic
```

加完之后断点正确落在 `kv.go:62`，`print string(key)` 得到 `"xxx"`。

⚠️ **陷阱预警 行号断点会随编辑漂移。** `break kv_test.go:39` 在你改动测试文件后可能指到别的语句上。稳定的做法是**函数名 + 条件**（如 `break db.(*KV).Get if string(key) == "xxx"`），行号断点只用于一次性的现场。

11.4. 关键点

📌 **核心要点** 调试测试的顺序是固定的三步：① 先看 `go test` 的失败行号；② 在失败的那一行打断点（停在这条断言执行之前，而不是实现函数入口）；③ 读这一行将会用到的所有变量。这样一次停下就能判断「是测试写错、还是实现错了」，不需要在实现里逐行单步。

- `dlv test` 自带 `-N -l`，不需要你手动关优化；只有走 `dlv exec` 路线时才要自己加。
- 断点位置越靠近断言，变量越全（入参、返回值、被测对象状态都在作用域内）。
- 条件断点比「停很多次再手动判断」便宜：测试里的循环与重复调用都会被过滤掉。

11.5. 变式检验

- **测试里用了子测试（`t.Run`）：**`dlv test ./internal/db -- -test.run 'TestKVBasic/case1'`，断点仍按文件行设。

- **测试并行**（`t.Parallel`）：停下后先 `goroutines`，再用 `goroutine <id>` 切到断点所在协程，否则看到的是别的测试的状态。
- **想看 panic 现场**：不要设断点，直接 `dlv test` 后 `continue`；进程崩溃时 Delve 会停在 panic 点，`bt` 给出完整栈。
- **想验证「同键同值 → updated=false」**：把断点设在 `kv_test.go:36` 之前的 `Set` 调用上，或直接 `break db.(*KV).Set if string(key) == "k1"`，看 `updated` 的返回值。

11.6. 同类归类

子命令	调试对象	何时用
`dlv test`	包里的 `_test.go`	本项目全部测试调试，首选
`dlv debug`	当前目录或指定包里的 `main`	写小 demo 验证 Go 语义
`dlv exec`	已编译的二进制	测试二进制复用、或调试别人的产物
`dlv attach`	正在运行的进程 PID	线上/长驻进程卡住
`dlv core`	core dump	崩溃后的事后分析
`dlv trace`	按正则给函数打 tracepoint，只输出不停下	想看调用序列、不想进会话

共性：它们都是「同一个调试内核 + 不同的程序来源」。所以 `break` / `print` / `locals` 这些命令在六种模式下完全一样，学会一套即可。

11.7. Go 语言机制补充

- **测试二进制**：`go test` 在包列表模式下会先编译出一个真正的可执行文件（内部调用 `cmd/go/internal/test`），再运行它；`go test -c` 只是把「编译」这一步单独暴露出来（`go help testflag` 有 `-c` 的说明）。Delve 调试的就是这个可执行文件，所以断点、变量、调用栈与普通程序完全一致。
- **`-gcflags` 的传参规则**：`go build` 的 `-gcflags '[pattern=]arg list'` 把参数转发给每次 `go tool compile` 调用；`all=` 表示对包及其全部依赖生效（`go help build` 的例子里用 `-gcflags=all=-S fmt` 打印反汇编）。只写 `-gcflags=-N` 时作用范围是命令行点名的包，依赖仍按优化编译。
- **`-N` / `-l` 的含义**：`/usr/lib/go/src/cmd/compile/internal/base/flag.go` 第 64 行与第 74 行分别定义这两个标志：

```
N      CountFlag "help:\"disable optimizations\""
LowerL CountFlag "help:\"disable inlining\""
```

关优化让变量始终落在栈槽上、关内联让函数边界保留——调试器依赖 DWARF 里的「变量在哪个位置」「这是哪个函数」这两类信息，两者被优化破坏后，`print` 就会给出垃圾值或 `unreadable`。

- **`-test.run` 是正则，不是名字精确匹配**：`go help testflag` 原文说明正则按 `/` 切分成多段，分别匹配子测试名；这也是 `-test.run 'TestKVBasic/case1'` 能选中子测试的原因。
- **为什么要有 `--allow-non-terminal-interactive`**：Delve 会检查 stdin 是否是终端；通过管道或 `--init` 驱动时不是终端，必须显式放行，否则它拒绝启动。本机原话：

```
Stdin is not a terminal, use '-r' to specify redirects for the target process
or --allow-non-terminal-interactive=true if you really want to specify a redirect
for Delve
```

11.8. 本次操作与当前状态

- 新增本节，回答「如何用 Delve 调试 `kv_test.go` 里的测试」。注意文件名是 `kv_test.go`，Go 的测试文件命名规则是 `*_test.go`，不存在 `testKv.go` 这种写法（那样会被当成普通源文件编译进包）。
- 本机环境已具备：`dlv` 1.27.2（`/home/zz/go/bin/dlv`）、`go` 1.27.3，二者版本匹配，`--check-go-version` 默认开启也通过。
- 本节所有命令与输出均在本机实跑：`dlv test` 断点、条件断点、`args` / `print`、`dlv exec` 优化与关闭优化两组对照、`dlv dap`、`--headless`。
- 未改动 `internal/db` 任何文件；调试用的 init 脚本与测试二进制写在会话临时目录，未进仓库。
- 本人可直接开始，命令如下：

```
cd /home/zz/project/db-go
dlv test ./internal/db -- -test.run TestKVBasic
```

进入会话后依次输入 `break kv_test.go:36`、`continue`、`next`、`print kv.mem`。

Index of Listings

Listing 1 UWCSE 331 的 IntSet：RI 与 add 里的检查必须成对 5
Listing 2 本机可直接验证的四个出处 7
Listing 3 第二版 AF 的现状 9
Listing 4 补全 AF 的两种写法 9
Listing 5 Safety 的承诺必须与实现一致 10
Listing 6 三个方法的最小契约 10
Listing 7 三个本步会用到的 Go 机制 11
Listing 8 标准库选择 ok bool：未命中不是错误 12
Listing 9 GOROOT 里的三处 Invariant 注释 13
Listing 10 6.102 原文里 max 函数的 strategy 写法 14
Listing 11 6.102 原文的 abs 例子 16
Listing 12 6.102 原文的 max 例子 16
Listing 13 标准库表驱动：注释标出维度，每个 case 是一个子域的代表 17
Listing 14 示意形态：冒号前是维度，冒号后是子域 17
Listing 15 当前内容（原文照录，含格式问题） 17
Listing 16 每个操作只按自己的行为分叉点切 18
Listing 17 三个 partition 的笛卡尔积是 12 组，7 条断言只覆盖其中一部分 20
Listing 18 partition on operation 的当前内容 21
Listing 19 标准库用函数名当子域标签 21
Listing 20 等待本人改文件时反复执行的轮询命令 23
Listing 21 operation 与 key state 两块的现状 26
Listing 22 Set 的 3 个子域，名字直接对应行为分叉 27
Listing 23 sync/map.go 的两个样本：键存在性一个 bool，值是否变了另一个 bool 27
Listing 24 history 的两种落法 28
Listing 25 dlv help test 的输出（本机 dlv 1.27.2） 30
Listing 26 进入交互式调试；– 之后是传给测试二进制的参数 30
Listing 27 把上面的命令存成 init 文件，用 –init 喂给 dlv 即可复现 31
Listing 28 第 39 行停下时的 val / ok / err / mem 31
Listing 29 只在键为 xxx 时停下；实测命中的是第三次调用 32